

Assignment 3 Report — Group 9

Brandon Tran (M.S. S&DS)
brandon.tran@yale.edu

Cailey Bobadilla (M.S. S&DS)
cailey.bobadilla@yale.edu

March 2, 2026

Note: All code for Assignment 3 can be found on GitHub.

1 Single-Model Calibration

The goal of this section is to explore the impact of prompt language and temperature. Prompt language refers simply to the words we use to describe a query for our large language model (LLM). Temperature, T , in this context is a hyperparameter that controls the randomness (or “creativity”) of the model’s predictions by scaling the probabilities of the next potential tokens. $T = 1$ instructs the model to sample from a natural, unscaled probability distribution. As $T \rightarrow 0$, the model becomes more deterministic, with the probability of the highest-scoring token approaching 1. Conversely, as $T \rightarrow \infty$, the differences between token scores shrink; the probability distribution becomes uniform such that every token has an equal chance of being selected.

1.1 Simple Scattergories Question: Day of the Week

Includes Reflection Question 1.

We began our investigation with queries to a single local Ollama model, `qwen2.5:7b`, which is trained on one of Alibaba’s large-scale datasets. Our goal was to see which prompt and temperature would lead the model could reliably produce a uniform distribution of valid days of the week. That is, out of 3,500 queries, we expected the model to output each day of the week 500 times.

We compared eight prompts in total and restate the four that yielded the most interesting results here:

1. Return a day of the week in lowercase. Output only the weekday (no additional text). For example, you would output: *saturday*
2. Select exactly one day of the week from this list: [*thursday, tuesday, saturday, sunday, wednesday, monday, friday*]. You must pick a day completely at random. Be entirely unpredictable. Output only the lowercase day name, nothing else.
3. Imagine you have a fair, 7-sided die. You roll it. If it lands on 1, the answer is *monday*. If it lands on 2, the answer is *tuesday*. If it lands on 3, the answer is *wednesday*. If it lands on 4, the answer is *thursday*. If it lands on 5, the answer is *friday*. If it lands on 6, the answer is *saturday*. If it lands on 7, the answer is *sunday*. Tell me the day that corresponds to the number you rolled. Output ONLY the lowercase day name. No punctuation, no explanation.
4. “Return a day of the week in lowercase. Output only the weekday (no additional text). Don’t forget that *friday* and *saturday* are also valid days! For example, you would output: *saturday*”

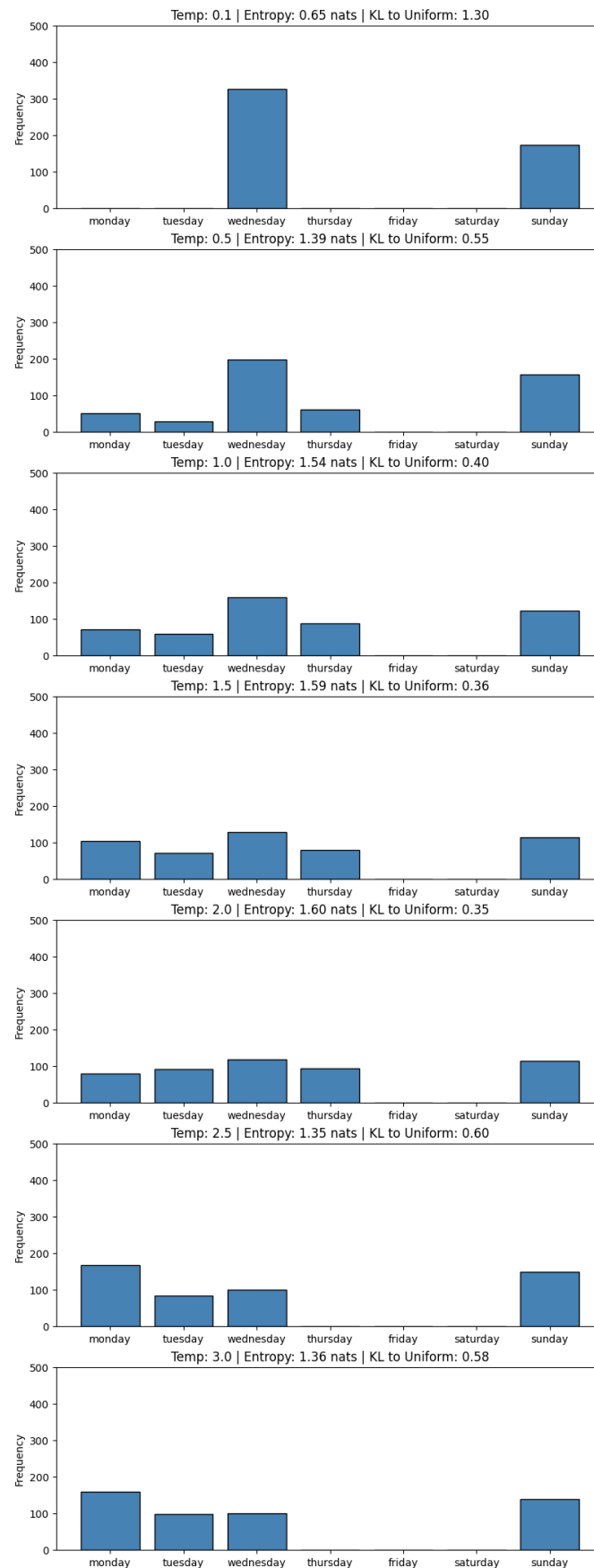


Figure 1: Prompt #1 performed best with temperature $T = 2.0$.

We found that, out of sheer luck, our first prompt (#1) yielded the best performance. Figure 1 illustrates the observed response frequency for each day of the week given different temperatures. Visually, we see the most even distribution of responses at temperature $T = 2.0$, which corresponds with the lowest KL divergence score from a Uniform distribution (0.35). As temperature continues to increase, we see a reversal in improvement, with Thursday falling entirely out of the domain of responses. Out of the eight prompts we ran full simulations with (500 queries per temperature per prompt), only Prompt #1 was able to yield these almost-Uniform results. We tried to recuperate the missing Friday and Saturday responses via Prompt # 4; however, this resulted in even poorer performance.

1.2 Next-Token Probabilities

Includes Reflection Question 2.

Controlling for the model (qwen2.5:7b), Top-K (which we set to $K = 40$ to allow for “creative” responses), and prompt (#1), we were able to uncover the next-token probabilities assumed for each token by the model via API calls. A maximum of 20 probabilities were provided in the form of logprobs, which were normalized and compared against observed frequencies under $T = 1.0$ in Figure 2 and Table 1.

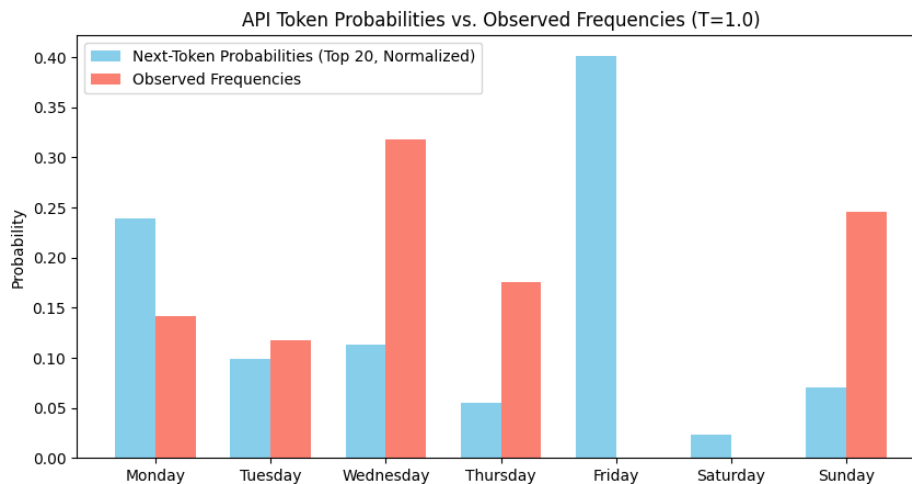


Figure 2: The model’s normalized next-token probabilities differed from the observed data, notably for *Friday*.

Day	Next-Token	Observed
Monday	0.2387	0.1420
Tuesday	0.0986	0.1180
Wednesday	0.1127	0.3180
Thursday	0.0551	0.1760
Friday	0.4015	0.0000
Saturday	0.0229	0.0000
Sunday	0.0705	0.2460

Table 1: qwen2.5:7b’s normalized next-token probabilities against observed frequencies at $T = 1.0$.

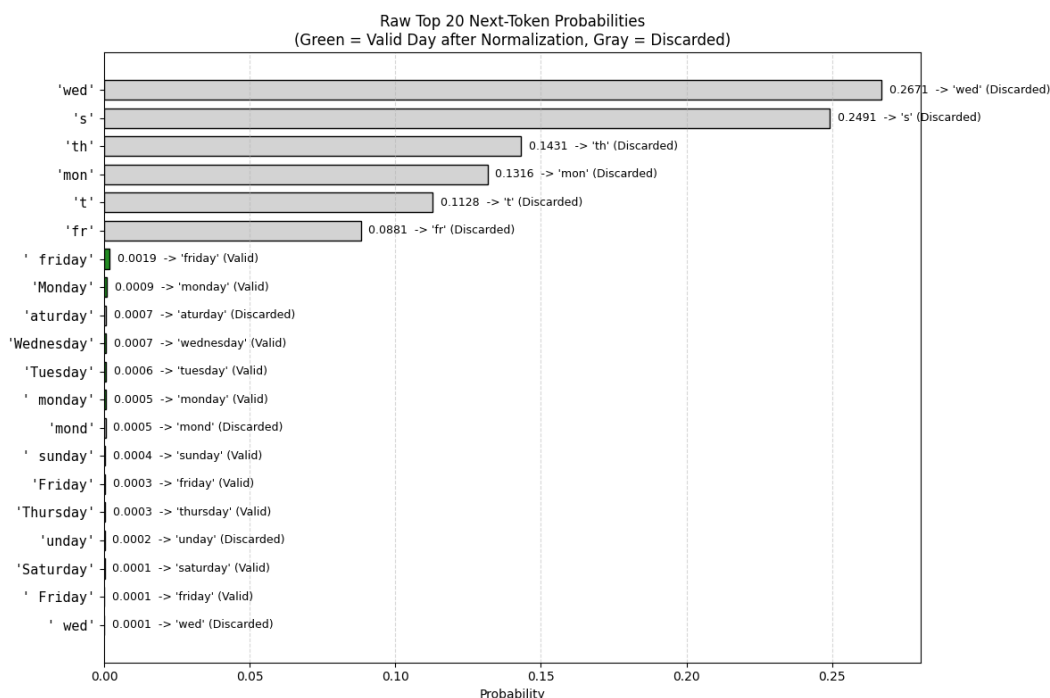


Figure 3: qwen2.5:7b’s next-token probabilities at $T = 1.0$ without normalization. Valid days that were included in Figure 2 are in green.

We compared the next-token probabilities against the observed frequencies at $T = 1.0$ because temperature acts as a divisor in the model’s softmax function ($P_i = \frac{\exp(z_i/T)}{\sum \exp(z_j/T)}$). At $T = 1.0$, the logits are unscaled, meaning the API’s returned probabilities perfectly represent the model’s base distribution. This allows for a fairer one-to-one comparison between the model’s statistical weights and its actual generative output before any manual temperature warping is applied.

Despite the fact that these numbers should match for each day, note in Table 1 that the probabilities and frequencies are disparate for all but Tuesday. This is due to the application of normalization to the next-token probabilities.

When reviewing the raw next-token probabilities without normalization, we observe the distribution seen in Figure 3. Now, we can infer top probabilities in the order of: Wednesday, Sunday (or Saturday), Thursday, Monday, Tuesday (or Thursday), and Friday, which much more closely resemble the observed frequencies from our sample. This observed discrepancy between the normalized probabilities and observed frequency is a result of tokenization. Recall that in our sampling procedure, we set `max_tokens = 8` to allow for 32 characters, or approximately one word. In just one token, the model rarely predicts a full day name; rather, a prefix like “wed” is much more likely and is what we observe in 3. Then, our sampling procedure allows for the model to complete “wed” into “Wednesday,” and outputs this as the final result, which is what we observe in Figure 2.

1.3 Scattergories Redux

Includes Reflection Questions 3–4.

With a basic understanding of the settings at play behind the output of our model in hand, we

proceeded to challenge qwen2.5:7b with a more complex task. Rather than naming days of the week, for which the support is limited, we asked the model to name fruits that start with the letter *b*. Maintaining our same experimental pipeline, we tested three prompts:

1. We are playing a game of Scattergories. The category is 'Fruits' and the letter is 'B'. What is a fruit that starts with the letter 'B'? Output only the lowercase word: no punctuation, no explanation. For example, 'banana'
2. What is a fruit that starts with the letter 'B'? Output only the lowercase word: no punctuation, no explanation. For example, 'banana'
3. What is a fruit that starts with the letter 'B'? Output only the lowercase word: no punctuation, no explanation. Do not say blueberry. For example, 'banana'

We visualize our results for Prompts #1 and 3 in Figure 4. We constructed Prompt #2 to test whether removing mentions of Scattergories would impact responses. Ultimately, we chose to omit these results, as the model failed to respond with any other term than *blueberry*. Interestingly, we see a vast increase in response diversity with Prompt #3, which was formulated specifically to counteract the fact that the model seemed to say *blueberry* by large for Prompt #1 (the line "do not say blueberry" was appended to the prompt). The model instead becomes dominated by *banana*, which we use as our example term, but allocates the remaining probability mass across along tail of lower-probability tokens, some of which do not properly answer the prompt (e.g., bababa, cherry, pear).

Temperature	Prompt 1 (Simple)	Prompt 3 (No Blueberry)
0.1	2	2
0.5	4	35
1.0	5	79
1.5	4	107
2.0	4	110
2.5	5	126
3.0	4	119

Table 2: Comparison of unique fruits generated across temperatures and prompts.

A comparison of unique terms in Table 2 further illustrates the vast growth in diverse responses. We may opt for explicitly constrained prompts when our goal is to force the model out of its deterministic comfort zone. Banning the highly probable token, *blueberry*, suppresses its raw logit. However, because the generation softmax function must sum to 1.0, this displaced probability mass is, by definition, reallocated down the long tail of the vocabulary distribution. When combined with higher temperatures, this logit reallocation drives the exponential increase in entropy seen in Figure 2. However, this strategy has a strict upper bound: as temperature scales beyond $T = 2.0$, the forced exploration of the long tail gives way to semantic collapse, causing the model to generate invalid fruits. Ultimately, this demonstrates that while prompt engineering can successfully expand a model's search space, temperature must be carefully calibrated to balance creative diversity against logical coherence.

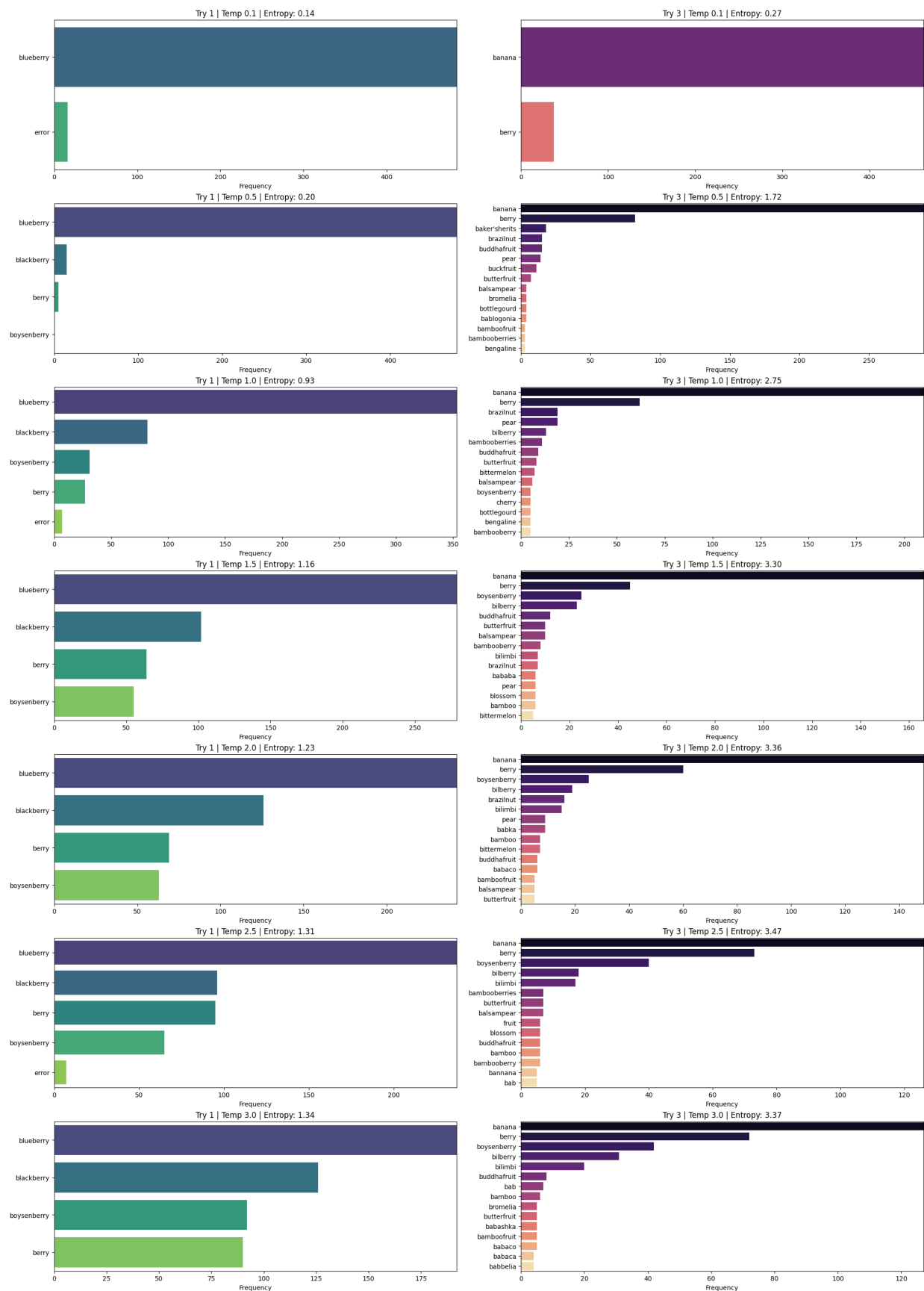


Figure 4: Prompt #3 incorporated much more diversity in responses than Prompt #1.

2 Self-Play with One Model

2.1 Model, Prompts, and Temperatures

For both Part 2 and Part 3, three scenarios were implemented for self-play and cross-play between models. The specific scenarios used were:

1. **Scenario 1:** Original Prompt, Temperature = 0.9
2. **Scenario 2:** Original Prompt, Temperature = 2.0
3. **Scenario 3:** Obscure Prompt, Temperature = 2.0

The specific prompts used were:

- **Original Prompt:** "You are playing a game of Scattergories. Name exactly one category that starts with the letter {letter}. Output only the answer, with no punctuation, explanation, or conversation."
- **Obscure Prompt:** "You are playing Scattergories. Name a highly obscure, rare, but valid {category} that starts with the letter {letter}. Your goal is to choose a correct answer that no other player will think of. Output only the answer, with no punctuation or explanation."

The model used for self-play in this section was `qwen2.5:7b`.

2.2 Audit Quality

To determine the accuracy and error rates of the judge (provided as `judge.py`), a random sample of 50 judged rows was taken and manually verified. These rows consisted of judged samples from Scenario 1 and Scenario 2 using the `qwen2.5:7b` model. The calculated rates were:

- **Estimated Judge Accuracy Rate:** 92.0%
- **Estimated Judge Error Rate:** 8.0%

2.3 Self-Play Experiments

Includes Reflection Question 4.

Table 3: Self-Play Experiment Outcomes across varying temperatures and prompts. For the easiest and hardest categories, the values in parentheses represent the (Average Points, Validity Rate).

Metric	Orig. Prompt, Temp = 0.9	Orig. Prompt, Temp = 2.0	Obs. Prompt, Temp = 2.0
Overall Validity	76.9%	73.1%	36.9%
Avg. Score/Player	32.8%	43.4%	28.7%
Continued on next page			

Table 3 – Continued from previous page

Metric	Orig. (0.9)	Orig. (2.0)	Obs. (2.0)
Top 3 Easiest	1. Q - Words (1.00, 100%)	1. Q - Words (1.00, 100%)	1. K - Grocery (0.90, 90%)
	2. W - Cities (1.00, 100%)	2. B - Singers (0.90, 90%)	2. Q - Words (0.80, 80%)
	3. G - Foods (1.00, 100%)	3. A - Jobs (0.90, 90%)	3. W - Park (0.80, 80%)
Top 3 Hardest	1. P - Figures (0.00, 100%)	1. F - Fish (0.00, 100%)	1. Z - Animals (0.00, 100%)
	2. O - Companies (0.00, 100%)	2. Z - Animals (0.00, 100%)	2. X - Brands (0.00, 0%)
	3. N - Hospital (0.00, 100%)	3. C - Countries (0.00, 100%)	3. B - Fruits (0.00, 0%)

Based on the results in Table 3, Scenario 1 (**Default, Temp 0.9**) achieved highest overall validity at 76.9%. However, Scenario 2 (**Optimized, Temp 2.0**) produced the highest average score per player at 43.4%. While validity was higher for Scenario 1, the score per player is a more significant metric for the game itself. Since validity only measures if an answer starts with the correct letter and fits the category, it does not account for the collisions that happen when both players give the same “safe” answer. The higher temperature in Scenario 2 introduces enough variance to reduce these collisions, leading to better overall scores despite the slight decrease in validity. Thus, a temperature of 2.0 is more desirable than 0.9 for maximizing game performance.

In contrast, Scenario 3 (**Obscure, Temp 2.0**) performed the worst across all metrics. The restrictive nature of the obscure prompt likely forced the model to prioritize rarity over accuracy, leading it to suggest answers that did not actually fit the categories. This resulted in significantly lower validity (36.9%) and a lower average score per player (28.7%) compared to the other two scenarios.

3 Cross-Model Competition

3.1 Models, Prompts, and Temperatures

For more information on the specific scenarios used and their temperatures and prompts, refer back to Subsection 2.1.

The models used for self-play and cross-play in this section were qwen2.5:7b, llama3.2:3b, and gemma2:2b.

3.2 Self-Play Analysis

Table 4: qwen2.5:7b Self-Play Outcomes across varying temperatures and prompts. For the easiest and hardest categories, the values in parentheses represent the (Average Points, Validity Rate).

Metric	Orig. Prompt, Temp = 0.9	Orig. Prompt, Temp = 2.0	Obs. Prompt, Temp = 2.0
Overall Validity	77.3%	76.2%	35.9%
Avg. Score/Player	34.2%	39.7%	28.7%
Top 3 Easiest	1. Q - Words (1.00, 100%) 2. G - Toolbox (1.00, 100%) 3. N - Birds (0.90, 90%)	1. B - Red things (1.00, 100%) 2. J - Refrigerator (1.00, 100%) 3. D - Plants (1.00, 100%)	1. X - Words (1.00, 100%) 2. K - Grocery (1.00, 100%) 3. Q - Words (0.80, 80%)
Top 3 Hardest	1. L - Library (0.00, 0%) 2. Q - Fict. chars (0.00, 100%) 3. I - Cities (0.00, 0%)	1. I - Cities (0.00, 0%) 2. N - Hospital (0.00, 100%) 3. M - Languages (0.00, 100%)	1. E - Companies (0.00, 0%) 2. Z - Animals (0.00, 100%) 3. A - Kitchen (0.00, 0%)

Table 5: llama3.2:3b Self-Play Outcomes across varying temperatures and prompts. For the easiest and hardest categories, the values in parentheses represent the (Average Points, Validity Rate).

Metric	Orig. Prompt, Temp = 0.9	Orig. Prompt, Temp = 2.0	Obs. Prompt, Temp = 2.0
Overall Validity	83.3%	82.2%	37.5%
Avg. Score/Player	38.0%	52.5%	31.2%
Top 3 Easiest	1. B - Fruits (1.00, 100%) 2. J - Refrigerator (1.00, 100%) 3. H - Countries (1.00, 100%)	1. Q - Words (1.00, 100%) 2. C - Desserts (1.00, 100%) 3. S - Animals (1.00, 100%)	1. G - Foods (0.90, 90%) 2. D - Plants (0.80, 80%) 3. Y - Foods (0.80, 80%)
Top 3 Hardest	1. A - Animals (0.00, 100%) 2. N - Birds (0.00, 100%) 3. L - Books (0.00, 100%)	1. V - Wear (0.00, 100%) 2. G - States (0.00, 100%) 3. F - Fish (0.00, 100%)	1. Y - Party (0.00, 0%) 2. R - Songs (0.00, 0%) 3. K - Superheroes (0.00, 0%)

Table 6: gemma2:2b Self-Play Outcomes across varying temperatures and prompts. For the easiest and hardest categories, the values in parentheses represent the (Average Points, Validity Rate).

Metric	Orig. Prompt Temp = 0.9	Orig. Prompt, Temp = 2.0	Obs. Prompt, Temp = 2.0
Overall Validity	74.7%	73.8%	35.0%
Avg. Score/Player	48.8%	54.7%	33.4%
Top 3 Easiest	1. E - Companies (1.00, 100%) 2. C - Countries (1.00, 100%) 3. M - Clothes (1.00, 100%)	1. A - Animals (1.00, 100%) 2. C - Board games (1.00, 100%) 3. P - Vegetables (1.00, 100%)	1. Y - Foods (1.00, 100%) 2. X - Words (1.00, 100%) 3. A - Animals (0.90, 90%)
Top 3 Hardest	1. I - Artists (0.00, 0%) 2. Y - Party (0.00, 0%) 3. G - States (0.00, 100%)	1. S - Office (0.00, 100%) 2. G - States (0.00, 100%) 3. B - Red things (0.00, 0%)	1. O - Companies (0.00, 0%) 2. K - Superheroes (0.00, 0%) 3. C - Board games (0.00, 0%)

As shown in Table 4, the overall validity and average scores per player are highly consistent with the results from Table 3. Because the model and scenarios are identical, this data serves as a baseline to ensure implementation consistency for the remaining self-play and cross-play analysis. For a more detailed discussion of these specific performance trends, refer back to Subsection 2.3.

For each of the models, findings remain consistent since overall validity is highest for Scenario 1 (original prompt, temperature = 0.9), but the average score per player Scenario 2 (original prompt, temperature = 2.0).

Among the models tested, llama3.2:3b achieved the highest overall validity at 83.3% in Scenario 1, while gemma2:2b reached the highest average score per player at 54.7% in Scenario 2. For self-play Scattergories, llama3.2:3b or gemma2:2b are preferable over qwen2.5:7b due to their higher and more competitive average scores. This performance gap may be linked to the increased inference time observed with these models, which can lead to more distinct and valid outputs.

Interestingly, there was no overlap in the top three easiest or hardest categories across all three models. The most notable categories appearing across different models were Q-Words (frequently easy) and K-Superheros (frequently hard).

3.3 Cross-Play Analysis

Table 7: Cross-Play Outcomes: Scenario 1 (Orig. Prompt, Temp = 0.9). For the easiest and hardest categories, values represent (Average Points, Validity Rate).

Metric	Qwen vs. Llama	Qwen vs. Gemma	Llama vs. Gemma
Overall Validity	78.4%	76.1%	76.1%
Overall Avg. Score	52.2%	60.5%	57.0%
Per-Model Breakdown	Qwen: 0.51 (77.5%) Llama: 0.53 (79.4%)	Qwen: 0.62 (77.2%) Gemma: 0.59 (75.0%)	Llama: 0.59 (78.4%) Gemma: 0.55 (73.8%)
Top 3 Easiest	1. A - Animals (1.00, 100%) 2. F - Backpack (1.00, 100%) 3. K - Cities (1.00, 100%)	1. A - Animals (1.00, 100%) 2. D - Instruments (1.00, 100%) 3. P - Figures (1.00, 100%)	1. Q - Words (1.00, 100%) 2. D - Plants (1.00, 100%) 3. O - Companies (1.00, 100%)
Top 3 Hardest	1. G - States (0.00, 100%) 2. Z - Animals (0.00, 100%) 3. X - Words (0.00, 100%)	1. I - Cities (0.00, 0%) 2. E - Sports (0.00, 100%) 3. G - States (0.00, 100%)	1. X - Brands (0.00, 100%) 2. Y - Party (0.00, 0%) 3. E - Sports (0.00, 100%)

Table 8: Cross-Play Outcomes: Scenario 2 (Orig. Prompt, Temp = 2.0). For the easiest and hardest categories, values represent (Average Points, Validity Rate).

Metric	Qwen vs. Llama	Qwen vs. Gemma	Llama vs. Gemma
Overall Validity	78.4%	74.7%	75.8%
Overall Avg. Score	57.5%	60.6%	57.7%
Per-Model Breakdown	Qwen: 0.56 (77.2%) Llama: 0.59 (79.7%)	Qwen: 0.63 (77.2%) Gemma: 0.58 (72.2%)	Llama: 0.62 (80.0%) Gemma: 0.53 (71.6%)
Top 3 Easiest	1. Q - Words (1.00, 100%) 2. G - Foods (1.00, 100%) 3. C - Board games (1.00, 100%)	1. Q - Words (1.00, 100%) 2. K - Cities (1.00, 100%) 3. M - Languages (1.00, 100%)	1. Q - Words (1.00, 100%) 2. W - Park (1.00, 100%) 3. A - Jobs (1.00, 100%)
Top 3 Hardest	1. Y - Party (0.00, 0%) 2. Z - Animals (0.00, 100%) 3. G - States (0.00, 100%)	1. E - Sports (0.00, 100%) 2. Z - Animals (0.00, 100%) 3. G - States (0.00, 100%)	1. H - Beach (0.00, 20%) 2. G - States (0.00, 100%) 3. Z - Animals (0.00, 100%)

Table 9: Cross-Play Outcomes: Scenario 3 (Obs. Prompt, Temp = 2.0). For the easiest and hardest categories, values represent (Average Points, Validity Rate).

Metric	Qwen vs. Llama	Qwen vs. Gemma	Llama vs. Gemma
Overall Validity	35.6%	32.7%	35.5%
Overall Avg. Score	34.4%	31.4%	34.8%
Per-Model Breakdown	Qwen: 0.33 (34.1%) Llama: 0.36 (37.2%)	Qwen: 0.32 (32.8%) Gemma: 0.31 (32.5%)	Llama: 0.38 (38.4%) Gemma: 0.32 (32.5%)
Top 3 Easiest	1. Q - Words (0.90, 90%) 2. Y - Foods (0.90, 90%) 3. X - Words (0.90, 90%)	1. Y - Foods (1.00, 100%) 2. Q - Words (0.80, 80%) 3. X - Words (0.80, 80%)	1. Q - Words (0.90, 90%) 2. D - Instruments (0.90, 90%) 3. Y - Foods (0.90, 90%)
Top 3 Hardest	1. Y - Party (0.00, 0%) 2. E - Sports (0.00, 0%) 3. L - Library (0.10, 10%)	1. F - Backpack (0.00, 0%) 2. M - Clothes (0.00, 0%) 3. Y - Party (0.00, 0%)	1. S - Office (0.00, 0%) 2. E - Companies (0.00, 0%) 3. Y - Party (0.00, 0%)

Across the pairwise comparisons in Table 7, Table 8, and Table 9, distinct performance trends emerge. The matchup between qwen2.5:7b and llama3.2:3b consistently achieves the highest overall validity across scenarios. However, the pairing of qwen2.5:7b and gemma2:2b yields the highest overall average score in two out of the three scenarios, likely due to a lower frequency of answer collisions between these two models.

In terms of individual model performance within these pairings, llama3.2:3b tends to secure more points on average than its competitors, and qwen2.5:7b generally outperforms gemma2:2b. Based on this trend of scoring efficiency and validity, the preference hierarchy for cross-play models is llama3.2:3b > qwen2.5:7b > gemma2:2b.

Regarding category-specific performance, Q-Words appears in the top three easiest categories in nearly every pairwise comparison, with the exception of the qwen2.5:7b vs. llama3.2:3b and qwen2.5:7b vs. gemma2:2b matchups in Scenario 1. The high scores in this category suggest the models have a diverse and valid vocabulary for this letter. Conversely, categories like E-Sports and Z-Animals appear as the hardest categories since the limited search space for those specific letter and category pairs often forces the models into either answer collisions or invalid hallucinations.

3.4 Summary

Includes Reflection Questions 5 -7.

Our analyses of self-play versus cross-play performance across Sections 2.3 and 3 consistently yielded higher average scores per player for cross-model outcomes. For example, in the respective order of qwen2.5:7b, llama3.2:3b, and gemma2:2b, recall that at $T = 2.0$, scores were 39.7%, 52.5%, and 54.7%, respectively. In comparison, cross-play scores were 60.6% between qwen and

gemma, 57.7% between llama and gemma, and 57.5% between qwen and llama. This performance gap implies that inter-model diversity surpasses intra-model diversity, which is unsurprising considering that each model is trained on separate sources of data and has different next-token probability distributions. In comparison, during self-play, a single model plays against its own training biases and probability distributions. Without high temperature forcing randomness, it is prone to generating the same high-probability answer for both Player 1 and Player 2, resulting in zero points. The inherent structural diversity enjoyed by cross-play reduces such collisions without requiring as much artificial randomness, resulting in higher overall scores.

This artificial randomness came from varied temperature settings, for which we tested $T = 0.9$, the somewhat “industry standard” for precise and accurate LLM responses, against $T = 2.0$, which we found in Part 1 to yield more creative and varied responses. In self-play, we saw temperature play a critical role. For example, llama3.2:3b observed a jump in score from 38.0% to 52.5% when temperature was increased, demonstrating a break from the repetition of the same outcome dictated by sampling from the same density without forcing variation. In cross-play, however, we did not observe such a noticeable change in performance. For example, pairing qwen with gemma resulted in only a 0.1% increase in score when the temperature was increased. We see that while temperature is key in enforcing diversity in self-play, diversity is intrinsically built in by varying our models, so the reduction in answer repetition that high temperature provides in the self-play setting is less valuable, if not negligible.

Ultimately, a combination of rudimentary prompt engineering and sampling settings are responsible for our observed performance gains. In Part 1, we found that adjustments to prompts were far more influential than changes in temperature settings. Some prompts generated invalid or homogeneous responses across all settings; it was not until we found a prompt that was well-understood by our model that we observed further variation (e.g., uniformity across the weekdays) across responses. As further illustrated in Parts 2 and 3, once we controlled for the ideal prompt (recall that our obscure prompt resulted in a steep drop in validity rates by up to $\approx 35\%$), temperature served as the primary driver in score improvement for self-play. Thus, we may think of this as a two-part strategy: we choose the proper vehicle for the race, then fine tune its tire selection to optimize for the specific track. We note that this is still a delicate balancing act: after observing our empirical distributions, we were able to modify our prompt to specifically exclude its first best guess (see Part 1.3), which resulted in more variation in responses without needing to fine-tune temperature.

4 Additional Reflection

Includes Reflection Questions 8-9.

The experiments conducted in this study suggest that AI systems are highly capable of interacting in the wild. As evidenced by the cross-play matches, all three models could successfully coordinate to play a structured game. However, because many models are trained on similar, overlapping datasets, their thought processes can be very similar.

Thus, when models are trained on the same data, they tend to converge on the same high-probability responses, causing their interactions to become redundant. For example, in Discord, some people implement custom bots specifically designed to mimic a person’s unique messaging style. While this is impressive, if multiple bots are trained on similar human data were to interact, they might eventually converge on the same generic patterns.

While these systems can interact today, the next few years of development will likely focus on making these conversations more advanced, distinct, and personalized. However, this growth brings concerns regarding safety and fairness. When AI agents interact, it is often difficult to see what is happening “behind the scenes.” If one model starts to hallucinate, it can contaminate the knowledge of the other models it is conversing with, leading to a cascade of misinformation that is difficult to stop or trace. Making sure these interactions remain transparent will be a major challenge as multi-agent systems become more common.

In this exercise, we adjusted primarily two levers: prompt and temperature. A natural continuation would be to incorporate personas, similar to what we did in Assignment 1 with *in silico* populations. Similar to prompt engineering, but via a more creative lens than we employed in the current experiment: if we ask an LLM to provide answers based on a certain persona, does this incorporate natural variation in its responses at the default model temperature (i.e., around $T = 0.9$)? Does this variation outcompete our current results? In the context of self-play, we hypothesize that at default temperatures, we will maintain high validity due to the lowered temperature, yet we will achieve lower collisions due to the different vocabulary used by each role. We presume that by asking the LLM to assume a given persona, the underlying next-token probability distribution will shift, and we posit that incorporating such personas better resembles how real-world humans—each with their own vocabulary, shaped by a myriad of factors, including age and interest—would play the game.

5 Disclaimer on Usage of AI

Cailey Bobadilla: I used Gemini 3 Pro and Claude Sonnet 4.6 for this report and the development of the Python scripts and Jupyter notebooks that generated the results in Section 2 and Section 3. Within the report itself, I used Gemini 3 Pro to convert the findings from the Jupyter notebooks into $\text{\LaTeX}$ tables. The detailed acknowledgements of each model's specific use cases are included in the respective scripts and notebooks.

Brandon Tran: Gemini Pro was crucial in debugging issues with the Yale GPU server to support parallel execution. It was able to incorporate `asyncio` into my original, serial code to execute Ollama queries concurrently, reducing wait times from 100 minutes to 10 minutes per query. Gemini also generally aided with code and troubleshooting and was helpful in generating visualizations and table for this report.